

Soutenance NYC Taxi Data Warehouse Snowflake & dbt

Script oral + déroulé de démonstration — durée maximale 10 minutes

Marie Antoinette Kilo

Objectif : commencer avec 5 slides très courtes, puis démontrer le projet réel : Snowflake, dbt Docs, GitHub Actions et Streamlit.

Plan de soutenance en 10 minutes

Temps	Support	Objectif	Message principal
0:00–0:45	Slide 1	Introduction	Présenter le but du pipeline.
0:45–1:45	Slide 2 + README	Architecture	RAW garde la donnée source, STAGING nettoie, FINAL sert aux analyses.
1:45–3:00	Snowflake	Qualité des données	48,7 M lignes RAW → 44,2 M trajets valides après nettoyage.
3:00–4:15	Snowflake / code	Transformations	Durée, vitesse, taux de pourboire et catégories.
4:15–5:30	dbt Docs	dbt et tests	Modèles versionnés, lineage et 37 tests PASS.
5:30–6:45	GitHub Actions	Automatisation	Pipeline mensuel, secrets sécurisés, notification en cas d'échec.
6:45–8:20	Streamlit	Dashboard	KPIs, graphiques, filtre WEEKDAY/WEEKEND.
8:20–9:30	README	Analyse réflexive	Difficultés, choix techniques, compétences et améliorations.
9:30–10:00	Conclusion	Bilan	Pipeline complet, testable et proche d'un contexte professionnel.

Script à lire pendant la présentation

0:00 - 0:45 | Slide 1 — Introduction

Bonjour. Je vais vous présenter mon projet de Data Engineering sur les données publiques des taxis jaunes de New York. L'objectif était de construire un pipeline complet : ingestion, nettoyage, transformations, tests, automatisation et dashboard.

J'ai travaillé sur les 12 mois de 2025, avec environ 48,7 millions de lignes au départ. En sortie, j'obtiens des tables analytiques fiables dans Snowflake, industrialisées avec dbt et visualisées avec Streamlit.

0:45 - 1:45 | Slide 2 — Architecture

À faire à l'écran : Afficher la slide architecture ou la section Architecture du README GitHub.

J'ai organisé le Data Warehouse en trois couches : RAW, STAGING et FINAL.

RAW garde les données brutes, sans transformation métier. STAGING sert à nettoyer, filtrer et enrichir les données. FINAL contient les tables d'analyse directement exploitables pour le dashboard.

Cette séparation me permet de garder une bonne traçabilité. Si une règle change, je peux repartir de RAW sans perdre la source originale.

Pour le chargement, j'ai utilisé un Internal Stage Snowflake. Dans un contexte professionnel, on pourrait remplacer cela par un External Stage connecté à S3, Azure Blob ou Google Cloud Storage.

1:45 - 3:00 | Démo Snowflake — RAW, nettoyage et qualité

À faire à l'écran : Aller dans Snowflake. Montrer la base NYC_TAXI_DB avec les schémas RAW, STAGING et FINAL.

À faire à l'écran : Montrer le volume RAW avec `SELECT COUNT(*) FROM NYC_TAXI_DB.RAW.YELLOW_TAXI_TRIPS;`

Résultat attendu : 48 722 602

Dans la couche RAW, j'ai 48 722 602 lignes. Avant de transformer les données, j'ai réalisé une analyse de qualité.

J'ai trouvé plusieurs anomalies : des montants négatifs, des distances à zéro ou extrêmement grandes, des dates incohérentes, des durées invalides et aussi des valeurs manquantes.

J'ai donc conservé uniquement les trajets de 2025, les distances supérieures à zéro et inférieures ou égales à 1000 miles, les durées positives et inférieures ou égales à 24 heures, et les montants financiers non négatifs.

Après nettoyage, je conserve 44 178 660 trajets. Cela représente un taux de rejet d'environ 9,33 %. Les anomalies peuvent se chevaucher, donc le taux est calculé sur les lignes rejetées distinctes.

3:00 - 4:15 | Slide 3 / Snowflake — Transformations et tables FINAL

À faire à l'écran : Montrer les colonnes enrichies ou le modèle `int_trip_metrics.sql`.

Après le nettoyage, j'ai enrichi les données. J'ai calculé la durée du trajet, la vitesse moyenne et le taux de pourboire.

J'ai aussi ajouté des catégories : la distance, la période de la journée et le type de jour, par exemple WEEKDAY ou WEEKEND.

À faire à l'écran : Montrer le schéma FINAL avec DAILY_SUMMARY, ZONE_ANALYSIS et HOURLY_PATTERNS.

Dans le schéma FINAL, j'ai créé trois tables analytiques. DAILY_SUMMARY analyse les trajets par jour. ZONE_ANALYSIS analyse les zones de prise en charge. HOURLY_PATTERNS analyse les comportements par heure et type de jour.

Ces tables contiennent les KPIs principaux : nombre de trajets, distance moyenne, durée moyenne, vitesse moyenne, chiffre d'affaires et pourboires.

4:15 - 5:30 | Slide 4 / dbt Docs — Industrialisation dbt

À faire à l'écran : Ouvrir dbt Docs, puis montrer le lineage.

J'ai industrialisé les transformations avec dbt Core. Le projet est organisé en trois niveaux : staging, intermediate et marts.

Le staging applique les règles principales de nettoyage. L'intermediate centralise les métriques calculées. Les marts produisent les tables finales utilisées par le dashboard.

dbt permet aussi de gérer automatiquement les dépendances entre les modèles avec ref(), de documenter les colonnes et de générer le lineage.

À faire à l'écran : Montrer les tests dbt ou le résultat 37 tests PASS.

J'ai ajouté des tests automatiques. Ils contrôlent les distances, les dates, les montants négatifs, les durées, les vitesses et certaines catégories. Au total, j'ai 37 tests qui passent sans erreur.

5:30 - 6:45 | Slide 4 / GitHub Actions — Automatisation

À faire à l'écran : Ouvrir GitHub Actions et montrer le workflow dbt Snowflake Pipeline avec Success.

Pour automatiser le pipeline, j'ai utilisé GitHub Actions. Le workflow peut être lancé manuellement et il est aussi planifié automatiquement chaque mois.

À chaque exécution, GitHub installe Python, installe les dépendances, crée le profil dbt à partir des secrets GitHub, vérifie la connexion Snowflake, exécute dbt build et génère la documentation.

Les informations sensibles, comme le mot de passe Snowflake, ne sont pas stockées dans le code. Elles sont gérées avec GitHub Actions Secrets.

J'ai aussi ajouté une notification : si le workflow échoue, une Issue GitHub est créée avec le lien vers l'exécution en erreur.

Point important : aujourd'hui, GitHub Actions automatise la partie dbt à partir de RAW déjà présent dans Snowflake. L'ingestion initiale des fichiers Parquet reste séparée. C'est une amélioration possible du projet.

6:45 - 8:20 | Slide 5 / Streamlit — Dashboard

À faire à l'écran : Ouvrir le dashboard Streamlit déjà lancé.

J'ai créé un dashboard avec Streamlit. Je l'ai choisi parce qu'il permet de construire rapidement un dashboard en Python et de se connecter directement à Snowflake.

À faire à l'écran : Montrer les quatre KPIs en haut du dashboard.

Le dashboard affiche les principaux KPIs : 44,2 millions de trajets, environ 1,276 milliard de dollars de chiffre d'affaires, 3,49 miles de distance moyenne et 17,6 minutes de durée moyenne.

À faire à l'écran : Descendre vers les graphiques : évolution quotidienne, trajets par heure, top 10 zones.

Ensuite, j'ai plusieurs visualisations : l'évolution quotidienne du nombre de trajets, le nombre de trajets par heure et le top 10 des zones de prise en charge.

À faire à l'écran : Changer le filtre Tous vers WEEKDAY puis WEEKEND.

J'ai aussi ajouté un filtre interactif pour comparer l'activité entre tous les jours, les jours de semaine et les week-ends. Le graphique horaire est recalculé automatiquement selon la sélection.

Une amélioration future serait d'ajouter le fichier officiel des zones NYC pour afficher les noms des zones au lieu des identifiants PULOCATIONID.

8:20 – 9:30 | Analyse réflexive

À faire à l'écran : Retourner sur le README, section Analyse réflexive.

Ce projet m'a permis de rencontrer plusieurs problèmes réels de Data Engineering.

Une première difficulté concernait les timestamps. Dans les fichiers Parquet, les dates étaient récupérées sous forme numérique. J'ai dû comprendre leur format et les convertir correctement avec Snowflake.

J'ai aussi rencontré un problème avec les vitesses très faibles qui devenaient zéro après arrondi. J'ai corrigé cela en conservant la précision dans les modèles intermédiaires et en réalisant l'arrondi seulement dans les tables finales.

J'ai également appris à structurer un projet dbt, gérer les schémas, écrire des tests, protéger les secrets et automatiser le pipeline avec GitHub Actions.

Pour aller plus loin, je pourrais automatiser toute l'ingestion mensuelle, ajouter des modèles incrémentaux, optimiser les coûts Snowflake et améliorer la partie géographique du dashboard.

9:30 – 10:00 | Conclusion

Pour conclure, j'ai construit un pipeline complet allant des fichiers Parquet jusqu'à un dashboard interactif.

Les données sont stockées et transformées dans Snowflake, les transformations sont industrialisées avec dbt, les tests valident la qualité, l'exécution est automatisée avec GitHub Actions et les résultats sont visualisés avec Streamlit.

Ce projet m'a permis de construire un pipeline structuré, testable, documenté et proche d'un contexte professionnel. Merci.

Déroulé de la démonstration — quoi ouvrir et dans quel ordre

1. PowerPoint : commencer avec les 5 slides pour poser le contexte.
2. README GitHub : montrer l'architecture et la documentation.
3. Snowflake : montrer NYC_TAXI_DB, les schémas RAW / STAGING / FINAL et les volumes.
4. dbt Docs : montrer le lineage et les tests.
5. GitHub Actions : montrer la dernière exécution Success.
6. Streamlit : montrer les KPIs, les graphiques et le filtre interactif.

Requêtes à préparer dans Snowflake

```
SELECT COUNT(*) FROM NYC_TAXI_DB.RAW.YELLOW_TAXI_TRIPS;  
SELECT COUNT(*) FROM NYC_TAXI_DB.STAGING.STG_YELLOW_TAXI_TRIPS;  
SELECT * FROM NYC_TAXI_DB.FINAL.DAILY_SUMMARY LIMIT 10;  
SELECT * FROM NYC_TAXI_DB.FINAL.ZONE_ANALYSIS ORDER BY TOTAL_TRIPS DESC LIMIT 10;  
SELECT * FROM NYC_TAXI_DB.FINAL.HOURLY_PATTERNS ORDER BY PICKUP_HOUR LIMIT 10;
```

À préparer avant de commencer

- Lancer Streamlit avant la soutenance : `streamlit run dashboard/app.py`
- Ouvrir Snowflake avec les schémas visibles.
- Ouvrir dbt Docs avec le lineage déjà affiché.
- Ouvrir GitHub Actions sur le dernier run en Success.
- Ne jamais afficher les secrets Snowflake ni le fichier `.streamlit/secrets.toml`.
- Garder le README ouvert pour servir de fil conducteur.

Réponses courtes aux questions possibles

Pourquoi trois couches RAW, STAGING et FINAL ?

RAW conserve les données sources, STAGING applique le nettoyage et les enrichissements, et FINAL contient les tables prêtes pour l'analyse métier. Cette séparation améliore la traçabilité et la maintenance.

Pourquoi dbt si tu avais déjà du SQL ?

SQL réalise les transformations, mais dbt permet de les organiser en modèles, de gérer les dépendances, d'ajouter des tests, de générer la documentation et de versionner le projet.

Pourquoi Streamlit ?

Streamlit permet de construire rapidement un dashboard interactif en Python et de se connecter directement à Snowflake. C'est adapté à un POC fonctionnel et versionné dans Git.

Pourquoi garder certaines valeurs NULL ?

Parce qu'elles ne concernent pas les colonnes critiques pour les KPIs. Je préfère conserver l'information manquante plutôt que d'inventer une valeur qui pourrait fausser les résultats.

Pourquoi 9,33 % de rejet ?

Ce taux correspond aux lignes qui ne respectent pas les règles de qualité : dates incohérentes, montants négatifs, distances invalides ou durées incorrectes. Les anomalies peuvent se chevaucher, donc je compte les lignes rejetées distinctes.

Est-ce que tout est automatisé ?

La partie dbt est automatisée : transformations, tests et documentation. L'ingestion initiale des fichiers Parquet vers RAW reste séparée. C'est une amélioration possible.

Que se passe-t-il si GitHub Actions échoue ?

Le workflow est marqué en erreur et une Issue GitHub est créée automatiquement avec le lien vers l'exécution en échec.

Quelle amélioration ferais-tu en premier ?

Automatiser complètement l'arrivée mensuelle des fichiers Parquet jusqu'à RAW, puis éventuellement utiliser des modèles dbt incrémentaux.

COMPLÉMENT AUX NOTES DE SOUTENANCE

Explication des 5 slides + démarrage après redémarrage + démo live + correction dbt Docs

Important : les 6 premières pages du document d'origine sont conservées. Les pages suivantes sont un complément pratique pour la soutenance.

A. Comment démarrer toute la démo après un redémarrage du PC

Prépare tout avant le début de la soutenance. Le but est de ne pas perdre de temps à lancer les outils devant le jury.

1. Ouvrir le projet et activer l'environnement Python

Ouvre une première fenêtre PowerShell puis exécute :

```
cd "C:\Users\Utilisateur\Desktop\cours\nyc-taxi-snowflake-dbt"  
.\env\Scripts\Activate
```

Tu dois voir (env) au début de la ligne PowerShell.

2. Lancer Streamlit - fenêtre PowerShell n°1

```
streamlit run dashboard/app.py
```

Le dashboard doit s'ouvrir dans le navigateur, généralement sur <http://localhost:8501>.

À ne pas faire : ne ferme pas cette fenêtre PowerShell pendant la soutenance, sinon le dashboard s'arrête.

3. Lancer dbt Docs - fenêtre PowerShell n°2

Ouvre une deuxième fenêtre PowerShell :

```
cd "C:\Users\Utilisateur\Desktop\cours\nyc-taxi-snowflake-dbt"  
.\env\Scripts\Activate  
cd nyc_taxi  
dbt docs generate  
dbt docs serve --port 8081
```

dbt Docs doit être accessible sur <http://localhost:8081>. Garde cette deuxième fenêtre PowerShell ouverte.

4. Préparer les autres onglets du navigateur

- README GitHub : positionné sur l'architecture RAW -> STAGING -> FINAL.
- Snowflake : base NYC_TAXI_DB visible avec les schémas RAW, STAGING et FINAL.

- dbt Docs : lineage déjà affiché.
- GitHub Actions : dernier run dbt Snowflake Pipeline ouvert en Success.
- Streamlit : dashboard chargé et fonctionnel.

Sécurité : ne montre jamais les valeurs des GitHub Secrets, le mot de passe Snowflake, profiles.yml ni .streamlit/secrets.toml.

5. Requêtes Snowflake à préparer avant de commencer

```
SELECT COUNT(*)
FROM NYC_TAXI_DB.RAW.YELLOW_TAXI_TRIPS;

SELECT COUNT(*)
FROM NYC_TAXI_DB.STAGING.STG_YELLOW_TAXI_TRIPS;

SELECT *
FROM NYC_TAXI_DB.FINAL.DAILY_SUMMARY
LIMIT 10;
```

Résultats à connaître : RAW = 48 722 602 lignes ; STAGING = 44 178 660 trajets conservés.

B. Explication détaillée des 5 slides - texte simple à lire

Slide 1 - NYC Taxi Data Warehouse

Ce que montre la slide : Le titre, les technologies principales et les chiffres clés : 48,7 M lignes RAW, 44,2 M trajets conservés et 37 tests dbt PASS.

Ce que je dis : Bonjour. Je vais vous présenter mon projet de Data Engineering sur les données publiques des taxis jaunes de New York. Mon objectif était de construire un pipeline complet, depuis les fichiers Parquet jusqu'à des tables analytiques et un dashboard. J'ai travaillé sur les 12 mois de 2025. Au départ, j'ai environ 48,7 millions de lignes brutes. Après nettoyage, je conserve environ 44,2 millions de trajets. Le projet utilise Snowflake, dbt Core, GitHub Actions et Streamlit.

À l'écran : Reste sur la slide. Ne commence pas encore à naviguer dans Snowflake.

Slide 2 - Architecture du Data Warehouse

Ce que montre la slide : Le chemin Parquet -> Internal Stage -> RAW -> STAGING -> FINAL et le rôle de chaque couche.

Ce que je dis : J'ai organisé le Data Warehouse en trois couches. RAW conserve les données brutes et sert de référence. STAGING applique les règles de nettoyage, les filtres et les enrichissements. FINAL contient les tables métier directement utilisées pour l'analyse et le dashboard. Cette séparation améliore la traçabilité : si une règle de nettoyage change, je peux repartir de RAW sans perdre la donnée source. Pour ce projet, j'ai utilisé un Internal Stage Snowflake pour charger les fichiers Parquet.

À l'écran : Avec la souris, suis simplement le chemin de gauche à droite ou de haut en bas.

Slide 3 - Qualité des données et transformations

Ce que montre la slide : Le taux de rejet 9,326 %, les règles de qualité et les enrichissements : durée, vitesse, taux de pourboire et catégories.

Ce que je dis : Avant de créer les indicateurs, j'ai analysé la qualité des données. J'ai trouvé des dates incohérentes, des distances nulles ou extrêmes, des montants négatifs et des durées invalides. J'ai donc défini des règles explicites : garder les trajets de 2025, une distance supérieure à zéro et inférieure ou égale à 1000 miles, une durée positive et inférieure ou égale à 24 heures, et des montants financiers non négatifs. Après nettoyage, le taux de rejet est d'environ 9,33 %. J'ai ensuite calculé la durée, la vitesse, le taux de pourboire et plusieurs catégories utiles pour l'analyse.

À l'écran : Montre les deux blocs : règles principales puis enrichissements.

Slide 4 - Industrialisation : dbt + GitHub Actions

Ce que montre la slide : Les niveaux staging/intermediate/marts, 37 tests PASS, la documentation dbt, GitHub Actions et la notification en cas d'échec.

Ce que je dis : J'ai utilisé dbt Core pour industrialiser les transformations SQL. Le staging nettoie les données, l'intermediate calcule les métriques et les marts produisent les tables finales. dbt gère les dépendances entre les modèles, les tests et la documentation. J'ai 37 tests qui passent sans erreur. J'ai ensuite automatisé cette partie avec GitHub Actions : le workflow peut être lancé manuellement et il est planifié chaque mois. Les identifiants Snowflake sont stockés dans GitHub Secrets. Si le workflow échoue, une Issue GitHub est créée automatiquement avec le lien vers le run.

À l'écran : Cette slide prépare la démo dbt Docs et GitHub Actions qui arrive juste après.

Slide 5 - Dashboard et bilan

Ce que montre la slide : Les 4 KPIs, les visualisations du dashboard et les améliorations futures.

Ce que je dis : Enfin, j'ai construit un dashboard Streamlit connecté aux tables FINAL de Snowflake. Il affiche environ 44,2 millions de trajets, 1,276 milliard de dollars de chiffre d'affaires, 3,49 miles de distance moyenne et 17,6 minutes de durée moyenne. On peut aussi analyser l'évolution quotidienne, les heures d'activité et les zones de prise en charge. Le dashboard possède un filtre pour comparer WEEKDAY et WEEKEND. Une amélioration future serait d'automatiser complètement l'ingestion mensuelle et d'ajouter les noms officiels des zones NYC.

À l'écran : À la fin de cette slide, annonce : « Maintenant, je vais vous montrer rapidement le projet réel. »

C. Démonstration live - quoi faire ET quoi dire

Objectif temps : la démo doit être courte et fluide. Ne cherche pas des fichiers pendant la présentation : tout doit être déjà ouvert.

Étape 1 - README / architecture

À faire : ouvre le README à la section Architecture et montre RAW -> STAGING -> FINAL.

À dire : « Ici, on retrouve l'architecture globale du projet. RAW conserve les données sources, STAGING contient les données nettoyées et enrichies, et FINAL contient les tables prêtes pour l'analyse métier. »

Étape 2 - Snowflake / RAW et STAGING

À faire : montre NYC_TAXI_DB et les trois schémas. Lance le COUNT de RAW puis le COUNT de STAGING.

À dire : « Dans RAW, j'ai 48 722 602 lignes sur les 12 mois de 2025. Après les règles de qualité, je conserve 44 178 660 trajets. Le taux de rejet est donc d'environ 9,33 %. Les anomalies peuvent se chevaucher, donc je calcule le rejet sur les lignes distinctes rejetées. »

Étape 3 - Snowflake / FINAL

À faire : montre DAILY_SUMMARY, ZONE_ANALYSIS et HOURLY_PATTERNS. Affiche quelques lignes de DAILY_SUMMARY.

À dire : « Dans FINAL, j'ai trois tables d'analyse. DAILY_SUMMARY analyse l'activité par jour, ZONE_ANALYSIS par zone de prise en charge, et HOURLY_PATTERNS par heure et type de jour. Ces tables sont plus simples à exploiter pour le reporting et le dashboard. »

Étape 4 - dbt Docs / lineage et tests

À faire : ouvre le lineage dbt. Montre RAW -> stg_yellow_taxi_trips -> int_trip_metrics -> les trois marts.

À dire : « Avec dbt, je peux voir le lineage complet du pipeline. Le staging applique le nettoyage, l'intermédiaire ajoute les métriques et les marts produisent les trois tables finales. J'ai également 37 tests dbt qui passent sans erreur. »

Conseil : ne passe pas du temps à ouvrir plusieurs tests pendant la démo. Montre un test documenté et le lineage ; c'est suffisant.

Étape 5 - GitHub Actions

À faire : ouvre le dernier run du workflow dbt Snowflake Pipeline et montre le statut Success.

À dire : « Le pipeline dbt est automatisé avec GitHub Actions. Il peut être lancé manuellement et il est aussi planifié chaque mois. Le workflow installe l'environnement, se connecte à Snowflake avec des secrets, exécute dbt build, les tests et la génération de la documentation. Si une étape échoue, une Issue GitHub est créée automatiquement. »

À ajouter si on te demande si tout est automatisé : « L'ingestion initiale des nouveaux fichiers Parquet vers RAW reste séparée. C'est l'une des améliorations futures du projet. »

Étape 6 - Streamlit

À faire : montre les quatre KPIs, puis les graphiques. Change le filtre Tous vers WEEKEND.

À dire : « Le dashboard interroge directement les tables FINAL de Snowflake. On retrouve les KPIs principaux, l'évolution quotidienne, l'activité par heure et les principales zones. Ici, je sélectionne

WEEKEND et le graphique est recalculé automatiquement. Cela montre que les tables finales peuvent directement alimenter une interface d'analyse. »

Étape 7 - Conclusion

À dire : « Pour conclure, j'ai construit un pipeline qui va des données brutes jusqu'à la visualisation. J'ai travaillé sur la qualité, la transformation, les tests, la documentation et l'automatisation. Le projet est fonctionnel, reproductible et proche d'un cas professionnel réel. Merci. »

D. Correction du problème dbt Docs : tests sans description

Le message « This test is not currently documented » vient du fait que les quatre tests personnalisés sont des tests dbt singuliers. Les commentaires placés dans le fichier SQL expliquent le code, mais ils ne deviennent pas automatiquement la description affichée dans dbt Docs.

Pour documenter un test singulier, dbt permet d'ajouter un fichier YAML dans le dossier tests. Crée le fichier suivant :

```
nyc_taxi/tests/tests.yml
```

Contenu recommandé :

```
version: 2

data_tests:
  - name: test_non_negative_amounts
    description: >
      Vérifie qu'aucun montant financier négatif ne subsiste dans les données
      nettoyées. Le test retourne les lignes invalides ; il passe si aucune ligne
      n'est retournée.

  - name: test_valid_dates
    description: >
      Vérifie la cohérence des dates des trajets après nettoyage, notamment
      leur appartenance à la période attendue et la cohérence chronologique
      entre le début et la fin du trajet.

  - name: test_valid_distance
    description: >
      Vérifie que les distances conservées respectent les règles de qualité
      du projet : distance strictement positive et non aberrante.

  - name: test_valid_trip_metrics
    description: >
      Vérifie la validité des métriques calculées pour les trajets, en particulier
      la durée et la vitesse, afin qu'aucune valeur invalide ne subsiste dans
      le modèle intermédiaire.
```

Ensuite, depuis le dossier nyc_taxi :

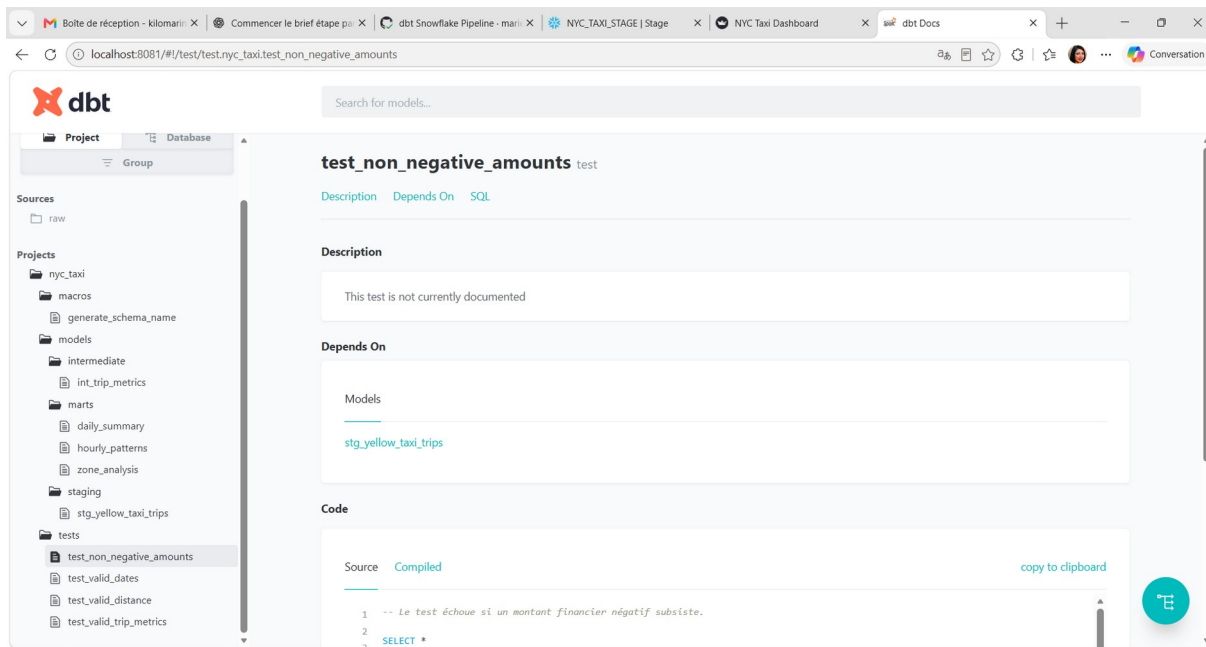
```
dbt parse
```

```
dbt test
dbt docs generate
dbt docs serve --port 8081
```

Recharge ensuite la page du test dans dbt Docs. La zone Description doit afficher le texte du fichier YAML.

Point important : le nom indiqué dans le YAML doit être exactement le nom du fichier SQL sans l'extension .sql.

Capture du problème observé



Dans dbt Docs, le test est bien présent mais sa description n'est pas encore déclarée dans un fichier YAML.

E. Est-ce que le README explique comment tout démarrer ?

Oui, en partie. Le README contient déjà l'installation locale, l'activation de l'environnement, l'exécution complète du projet, la génération de dbt Docs et la commande de lancement de Streamlit.

Ce qui manque surtout pour une soutenance est une section unique « Démarrage rapide après redémarrage du PC » qui regroupe les commandes dans l'ordre, avec deux terminaux séparés. Cela évite de chercher les commandes dans plusieurs sections du README.

Section recommandée à ajouter au README

```
## Démarrage rapide après redémarrage du PC

### 1. Activer l'environnement
```powershell
cd "C:\Users\Utilisateur\Desktop\cours\nyc-taxi-snowflake-dbt"
.\env\Scripts\Activate
```
```

2. Lancer le dashboard Streamlit

```
``powershell
streamlit run dashboard/app.py
``
```

Dashboard : `http://localhost:8501`

3. Lancer dbt Docs dans un deuxième terminal

```
``powershell
cd "C:\Users\Utilisateur\Desktop\cours\nyc-taxi-snowflake-dbt"
.\env\Scripts\Activate
cd nyc_taxi
dbt docs generate
dbt docs serve --port 8081
``
```

dbt Docs : `http://localhost:8081`

4. Ouvrir les services web

- Snowflake : afficher `NYC\_TAXI\_DB` et les schémas `RAW`, `STAGING`, `FINAL`.
- GitHub Actions : ouvrir le dernier run `dbt Snowflake Pipeline` en `Success`.

> Ne jamais versionner ni afficher `profiles.yml`, `.streamlit/secrets.toml`
> ou les valeurs des GitHub Secrets.

F. Checklist finale - 2 minutes avant la soutenance

- PowerPoint ouvert sur la slide 1.
- README ouvert sur l'architecture.
- Snowflake connecté avec NYC_TAXI_DB visible.
- Les requêtes COUNT sont déjà écrites dans le worksheet.
- dbt Docs ouvert sur le lineage.
- Un test dbt personnalisé affiche maintenant sa description.
- GitHub Actions ouvert sur le dernier run Success.
- Streamlit ouvert avec les KPIs visibles.
- Le filtre WEEKDAY/WEEKEND fonctionne.
- Aucun secret ou mot de passe n'est visible à l'écran.
- Chronomètre : rester sous 10 minutes.

Phrase de secours si une page met du temps à charger : « J'ai également la capture et la documentation dans le README, mais le service fonctionne normalement en local. Je poursuis avec l'étape suivante pour respecter les 10 minutes. »